

BCAC 0026: ANALYSIS OF ALGORITHM AND DATA STRUCTURES

Faculty Name

Dr. Vinod Jain, Assistant Professor, CEA

Mob : 9813078438, Email – vinod.jain@gla.ac.in

Seating : 329-B, AB-X, Third Floor

Course Structure

Credits: 04

L-T-P: 4-0-0

Module No.	Content	Teaching Hours
I	Introduction: Basic Design and Analysis techniques of Algorithms, time and space complexity, Correctness of Algorithm. Algorithm Design Techniques: Iterative techniques, Divide and Conquer, Dynamic Programming, Greedy Algorithms. Sorting Techniques: Elementary sorting techniques-Bubble Sort, Insertion Sort, Merge Sort, Advanced Sorting techniques-Heap Sort, Quick Sort, Sorting in Linear Time-Bucket Sort, Radix Sort and Count Sort. Searching Techniques and Complexity Analysis: Linear and Binary search, Medians & Order Statistics. Arrays: Single and Multi-dimensional Arrays, Sparse Matrices.	30
II	Stacks and Queues: Implementing stack using array and linked list, Prefix, Infix and Postfix expressions, Utility and conversion of these expressions from one to another; Array and Linked representation of Queue, De-queue, Priority Queues. Linked Lists: Singly, Doubly and Circular Lists, representation of Stack and Queue as Linked Lists. Recursion: Developing Recursive Definition of Simple Problems and their implementation; Advantages and Limitations of Recursion. Trees: Introduction to Tree as a data structure; Binary Trees, Binary Search Tree, (Creation, and Traversals of Binary Search Trees).	30

Recursion: Syllabus

- Recursion:
 1. Developing Recursive Definition of Simple Problems and their implementation
 2. Advantages and Limitations of Recursion.

RECURSION

RECURSION

RECURSION

RECURSION

RECURSION

RECURSION

RECURSION

Recursion

- “Calling a function by itself is called recursion”
- The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function.

```
void recurse()
```

```
{
```

```
... ..
```

```
recurse();
```

```
... ..
```

```
}
```

Recursive
Call



Recursion

- **Example 1 : Sum of Natural Numbers**
- Sum = $0 + 1 + 2 + 3 + 4 + \text{-----}$

Recursion

- **Example 1 : Sum of Natural Numbers**
- Sum = $0 + 1 + 2 + 3 + 4 + \text{-----}$

approach(1) – Simply adding one by one

$$f(n) = 0 + 1 + 2 + 3 + \text{.....} + n$$

Recursion

- **Example 1 : Sum of Natural Numbers**
- Sum = 0 + 1 + 2 + 3 + 4 + -----

approach(2) – Recursive adding

$$f(n) = 0 \qquad n=0$$

$$f(n) = n + f(n-1) \quad n \geq 1$$

Steps for recursion

- *Step1 – Define a base case:*
- *Step2 – Define a recursive case:*

Steps for recursion

- *Step1 – Define a base case:*
- *Step2 – Define a recursive case:*
- *Step3 – Ensure the recursion terminates:*
- *Step4 – Combine the solutions:*

Steps for recursion

- *Step1 – Define a base case: Identify the simplest (or base) case for which the solution is known or trivial. This is the stopping condition for the recursion, as it prevents the function from infinitely calling itself.*

Step2 – Define a recursive case: Define the problem in terms of smaller subproblems. Break the problem down into smaller versions of itself, and call the function recursively to solve each subproblem.

Steps for recursion

- *Step3 – Ensure the recursion terminates: Make sure that the recursive function eventually reaches the base case, and does not enter an infinite loop.*
- *Step4 – Combine the solutions: Combine the solutions of the subproblems to solve the original problem.*

// Recursive method to calculate factorial

```
static int factorial(int n) {
```

```
    if (n == 0 || n == 1) {
```

```
        return 1; // Base case
```

```
    } else {
```

```
        return n * factorial(n - 1); // Recursive call
```

```
    }
```

```
}
```

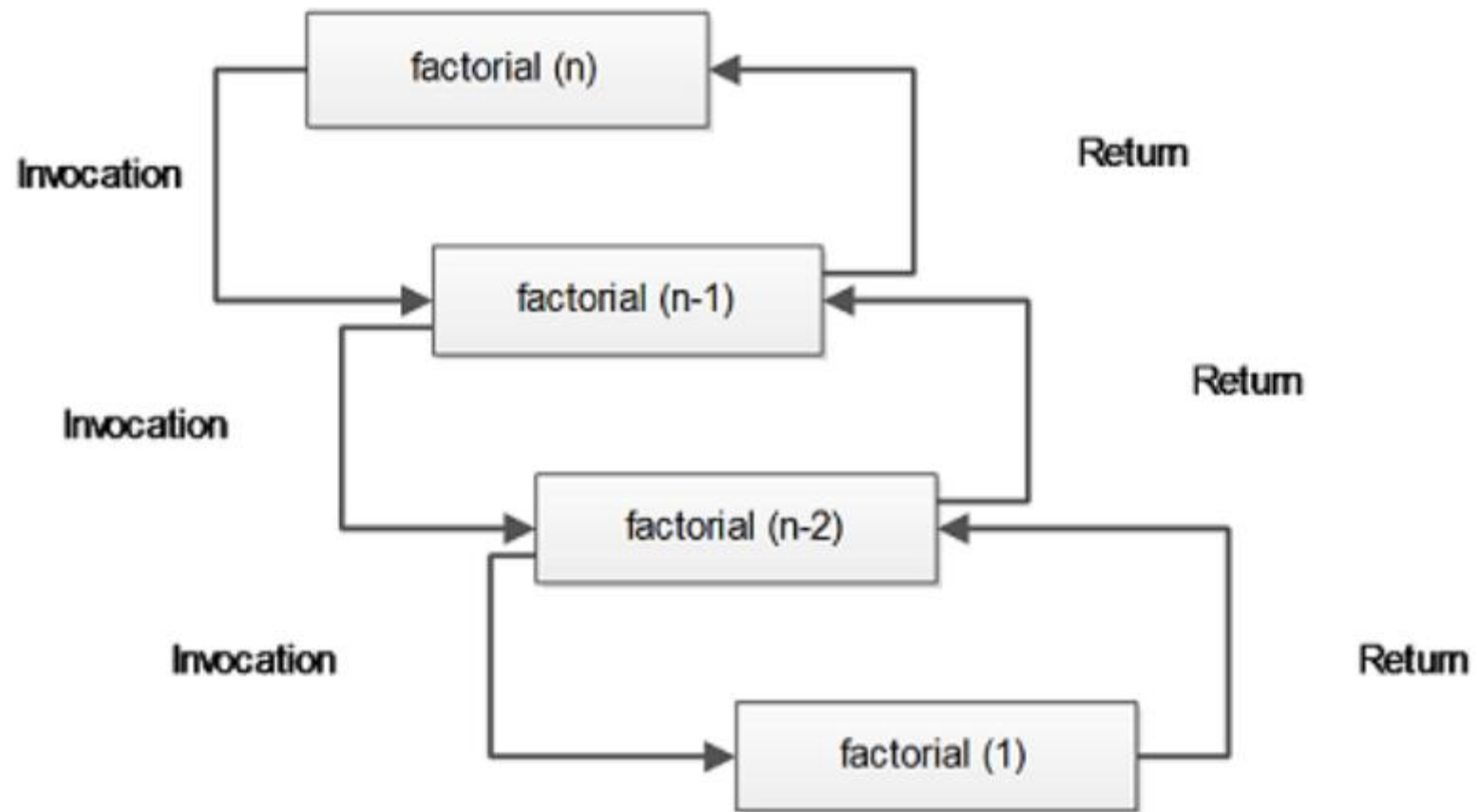
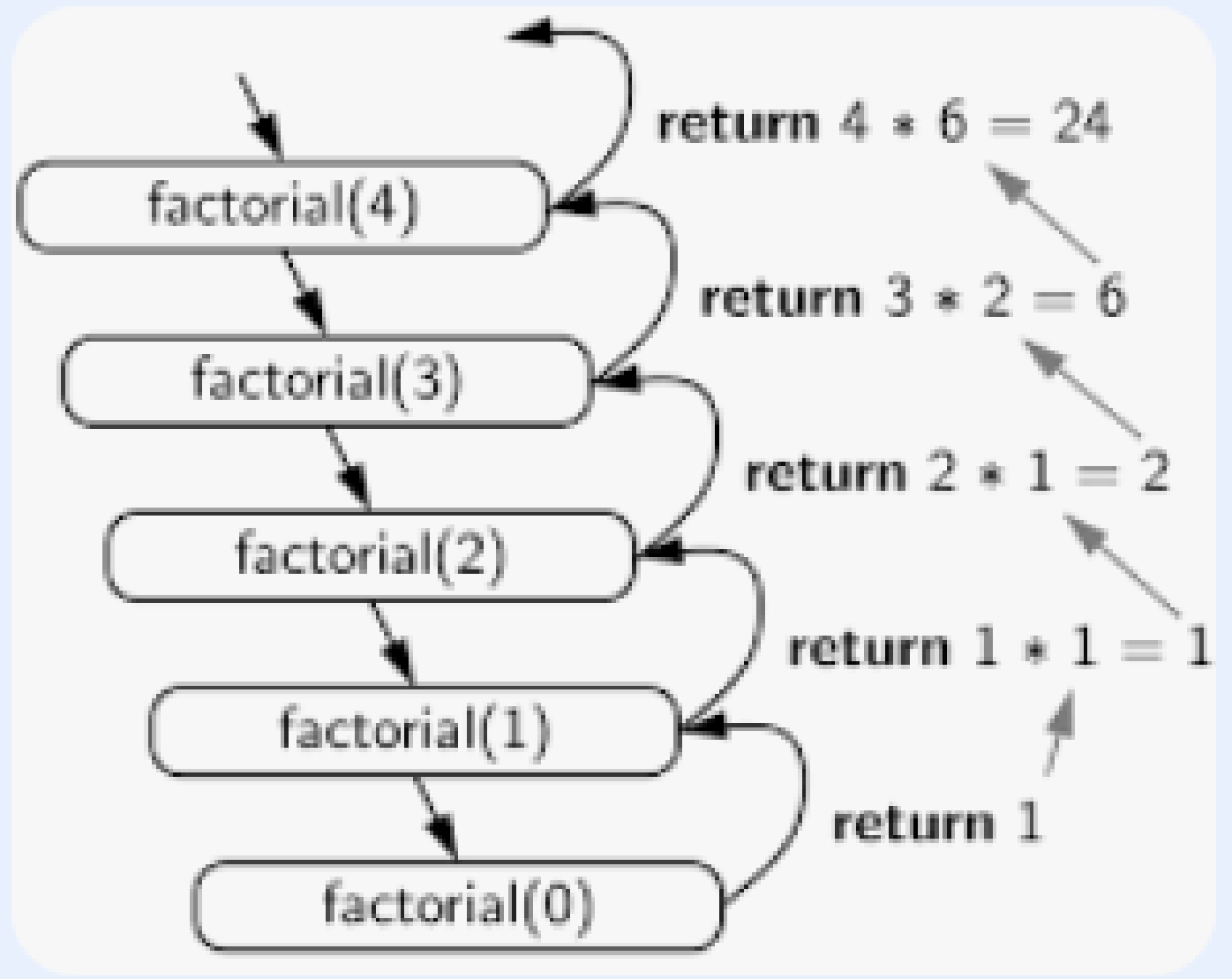


Figure 1 – Recursion




```
// factorial using recursion
import java.util.Scanner;
public class Factorial {
    // Recursive method to calculate factorial

static int factorial(int n) {
    if (n == 0 || n == 1)
    {
        return 1; // Base case
    }
    else
    {
        return n * factorial(n - 1);
        // Recursive call
    }
}
```

```
public static void main(String[] args)
{
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter a number: ");
    int number = scanner.nextInt();

    int result = factorial(number);

    System.out.println("Factorial = " + result);
}
}
```

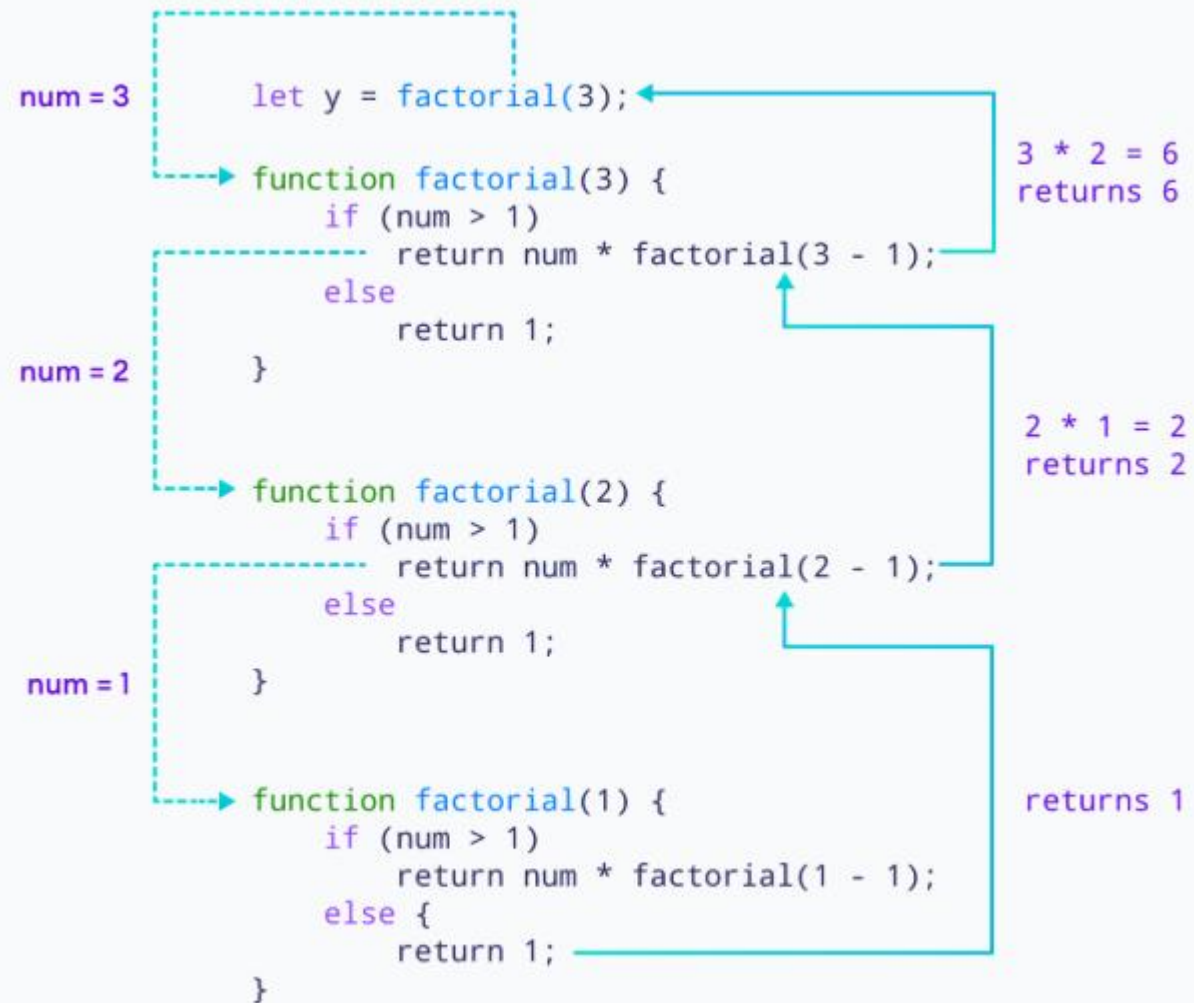
```
// factorial using recursion
public class Factorial {
    // Recursive method to calculate factorial

    static int factorial(int n) {
        if (n == 0 || n == 1)
        {
            return 1; // Base case
        }
        else {
            return n * factorial(n - 1); // Recursive call
        }
    }

    public static void main(String[] args) {
        System.out.println("Factorial OF 5 = " + factorial(5));
    }
}
```

run:

Factorial OF 5 = 120
BUILD SUCCESSFUL



Recursion

- **Java program** to calculate the **sum of first N natural numbers** using recursion:

Recursion

- **Example 1 : Sum of Natural Numbers**
- Sum = 0 + 1 + 2 + 3 + 4 + -----

approach(2) – Recursive adding

$$f(n) = 0 \qquad n=0$$

$$f(n) = n + f(n-1) \quad n \geq 1$$

Recursion

- Define the Recursive Function:
- If $n == 0$, return 0 (base case).
- Otherwise, return $n + \text{sum}(n - 1)$.

```
import java.util.Scanner;
public class SumOfNumbers {
    // Recursive method to calculate sum of first n natural numbers
    static int sum(int n) {
        if (n == 0) {
            return 0; // Base case
        } else {
            return n + sum(n - 1); // Recursive call
        }
    }
}
```

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    // Take input from the user  
  
    System.out.print("Enter a number: ");  
    int number = scanner.nextInt();  
    int result = sum(number);  
  
    // Display the result  
    System.out.println("For N = " + number);  
    System.out.println("Sum = " + result);  
    scanner.close();  
}  
}
```


Recursion

- A

Recursion

- **What is the base condition in recursion?**

A recursive program stops at a base condition. There can be more than one base conditions in a recursion. In the above program, the base condition is when $n = 1$.

- **How a particular problem is solved using recursion?**

The idea is to represent a problem in terms of one or more smaller problems, and add one or more base conditions that stop the recursion.

Recursion

- **When does Stack Overflow error occur in recursion?**
If the base case is not reached or not defined, then the stack overflow problem may arise. Let us take an example to understand this.

```
int fact(int n)
{
    // wrong base case (it may cause
    // stack overflow).
    if (n == 100)
        return 1;

    else
        return n*fact(n-1);
}
```

- If fact(10) is called, it will call fact(9), fact(8), fact(7), and so on but the number will never reach 100. So, the base case is not reached.
- If the memory is exhausted by these functions on the stack, it will cause a stack overflow error.

Recursion

- **What is the difference between direct and indirect recursion?**
- A function fun is called direct recursive if it calls the same function fun.
- A function fun is called indirect recursive if it calls another function say fun_new and fun_new calls fun directly or indirectly.

```
// An example of direct recursion
void directRecFun()
{
    // Some code....

    directRecFun();

    // Some code...
}
```

```
// An example of indirect recursion
```

```
void indirectRecFun1()
```

```
{
```

```
    // Some code...
```

```
    indirectRecFun2();
```

```
    // Some code...
```

```
}
```

```
void indirectRecFun2()
```

```
{
```

```
    // Some code...
```

```
    indirectRecFun1();
```

```
    // Some code...
```

```
}
```

Recursion VS Iteration

SR No.	Recursion	Iteration
1)	Terminates when the base case becomes true.	Terminates when the loop condition becomes false.
2)	Logic is built in terms of smaller problems.	Logic is built using iterating over something.
3)	Every recursive call needs extra space in the stack memory.	Every iteration does not require any extra space.
4)	Smaller code size.	Larger code size.

Recursion

- **What are the advantages of recursive programming over iterative programming?**
- Recursion provides a clean and simple way to write code.
- Some problems are inherently recursive like tree traversals, Tower of Hanoi, Merge Sort, Quick Sort etc.
- For such problems, it is preferred to write recursive code.

Recursion

- **What are the disadvantages of recursive programming over iterative programming?**

Recursive programs typically have more space requirements and also more time to maintain the recursion call stack.

- Recursion can make the code more difficult to understand and debug, since it requires thinking about multiple levels of function calls..

Recursion

- **Summary of Recursion:**
- There are two types of cases in recursion i.e. recursive case and a base case.
- The base case is used to terminate the recursive function when the case turns out to be true.
- Each recursive call makes a new copy of that method in the stack memory.
- Infinite recursion may lead to running out of stack memory.
- Examples of Recursive algorithms: Merge Sort, Quick Sort, Tower of Hanoi, Fibonacci Series, Factorial Problem, etc.

Recursion

- Here are some of the common applications of recursion:
 1. Tree and graph traversal:
 2. Sorting algorithms:
 3. Divide-and-conquer algorithms:
 4. Fractal generation:
 5. Backtracking algorithms:
 6. Memoization:

Recursion

- Here are some of the common applications of recursion:
- **Tree and graph traversal:** Recursion is frequently used for traversing and searching data structures such as trees and graphs.
- Recursive algorithms can be used to explore all the nodes or vertices of a tree or graph in a systematic way.
- **Sorting algorithms:** Recursive algorithms are also used in sorting algorithms such as quicksort and merge sort.

Recursion

- **Divide-and-conquer algorithms:**
- Many algorithms that use a divide-and-conquer approach, such as the binary search algorithm, use recursion to break down the problem into smaller subproblems.
- **Fractal generation:**
- Fractal shapes and patterns can be generated using recursive algorithms. For example, the Mandelbrot set is generated by repeatedly applying a recursive formula to complex numbers.

Recursion

- **Backtracking algorithms:** Backtracking algorithms are used to solve problems that involve making a sequence of decisions, where each decision depends on the previous ones. These algorithms can be implemented using recursion to explore all possible paths and backtrack when a solution is not found.
- **Memoization (Dynamic Programming DP) :** Memoization is a technique that involves storing the results of expensive function calls and returning the cached result when the same inputs occur again. Memoization can be implemented using recursive functions to compute and cache the results of subproblems.

Recursion

- A